

NumPy & Pandas workflows

COMPLETE LEARNING GUIDE

What You'll Learn

This submodule goes beyond the fundamentals to cover the workflows that show up constantly once you're actually manipulating real, messy, multi-table datasets: how NumPy's vectorized array operations and broadcasting rules let you avoid explicit loops entirely, how boolean indexing lets you filter data declaratively, and how pandas' groupby/aggregation and merge/join tools let you reshape and combine data the way you would with SQL, but interactively. These are the operations that fill the majority of real exploratory-data-analysis and feature-engineering time, and mastering them is what separates 'I can technically write pandas code' from 'I can reshape any dataset into the form I need quickly.' Every concept is demonstrated with runnable code on realistic small datasets.

1. NumPy Arrays and Vectorized Operations

A NumPy array is a fixed-type, contiguous block of memory that supports vectorized operations — applying a mathematical operation to every element at once, in compiled C code, rather than looping in Python. This is both far faster (often 10-100x) and far more concise than the equivalent explicit loop: `array + 5` adds 5 to every element in one expression, `array1 * array2` multiplies element-wise, and comparison operators (`array > 10`) produce a boolean array in one step. Internalizing 'operate on the whole array, don't loop over elements' is the single biggest mental shift moving from plain Python to NumPy/pandas.

PYTHON

```
import numpy as np

prices = np.array([19.99, 45.50, 12.00, 89.99, 5.25])

discounted = prices * 0.9          # vectorized: 10% off every price at once
expensive_mask = prices > 20       # boolean array: [False, True, False,
True, False]
expensive_prices = prices[expensive_mask] # boolean indexing: filter using that
mask

print(discounted)
print(expensive_prices) # [45.5  89.99]
```

2. Broadcasting: Operating on Arrays of Different Shapes

Broadcasting is the rule set NumPy uses to apply operations between arrays of different (but compatible) shapes without you having to manually reshape or repeat data. The rule, roughly: dimensions are compared from the right, and two dimensions are compatible if they're equal or one of them is 1 — a size-1 dimension is conceptually 'stretched' to match. This is what makes `array_2d - array_2d.mean(axis=0)` work directly (subtracting a 1D array of column means from every row of a 2D array) without writing an explicit loop over rows — an extremely common pattern for centering/normalizing data before feeding it into a model.

PYTHON

```
import numpy as np

scores = np.array([
    [80, 90, 70],
    [60, 85, 95],
    [75, 70, 88],
])

column_means = scores.mean(axis=0)      # shape (3,) – mean per column
centered = scores - column_means        # broadcasting: (3,3) - (3,) works
# automatically
print(column_means) # [71.67 81.67 84.33]
print(centered)
```

3. Boolean Indexing and Filtering in Pandas

Pandas extends NumPy's boolean indexing to DataFrames, and it's the standard way to filter rows declaratively — the pandas equivalent of a SQL WHERE clause. A boolean condition on a column (`df['amount'] > 100`) produces a boolean Series the same length as the DataFrame, which can then be used inside square brackets to keep only the matching rows. Combining conditions requires `&` (and) / `|` (or) rather than Python's `and`/or keywords, and each condition must be wrapped in parentheses because of operator precedence — a very common source of syntax errors for people new to pandas.

PYTHON

```
import pandas as pd

df = pd.DataFrame({
    "customer": ["A", "B", "C", "D"],
    "amount": [120, 45, 300, 90],
    "region": ["North", "South", "North", "South"],
})

high_value_north = df[(df["amount"] > 100) & (df["region"] == "North")]
print(high_value_north) # only customer C
```

4. GroupBy and Aggregation

`df.groupby('column')` splits a DataFrame into groups sharing the same value in that column, and is almost always followed by an aggregation that collapses each group into a summary — `.sum()`, `.mean()`, `.count()`, or `.agg()` for multiple statistics at once, optionally on different columns simultaneously. This 'split-apply-combine' pattern (split into groups, apply a function to each, combine the results back into one table) is the pandas equivalent of a SQL GROUP BY, and is the single most-used tool for turning row-level transactional data into summary-level insight.

PYTHON

```
import pandas as pd

df = pd.DataFrame({
    "region": ["North", "North", "South", "South", "South"],
    "amount": [120, 80, 300, 90, 150],
    "units": [2, 1, 5, 2, 3],
})

summary = df.groupby("region").agg(
    total_amount=("amount", "sum"),
    avg_units=("units", "mean"),
    order_count=("amount", "count"),
)
print(summary)
```

5. Merging and Joining DataFrames

Real data lives in multiple related tables, and `pd.merge()` combines them the way a SQL JOIN does, based on a shared key column. The `how` parameter controls join type: 'inner' keeps only rows with a match in both tables, 'left' keeps every row from the left table (filling unmatched right-side columns with NaN), 'right' is the mirror image, and 'outer' keeps every row from both sides. Choosing the wrong join type is a common, silent source of bugs — an inner join can quietly drop rows you expected to keep, while a left join can introduce unexpected NaN values you need to explicitly handle afterward.

PYTHON

```
import pandas as pd

orders = pd.DataFrame({"order_id": [1, 2, 3], "customer_id": [101, 102, 101]})
customers = pd.DataFrame({"customer_id": [101, 102, 103], "name": ["Aisha",
    "Rohan", "Meera"]})

# left join: keep every order, even if (hypothetically) the customer were missing
merged = orders.merge(customers, on="customer_id", how="left")
print(merged)

# Meera (103) has no orders and correctly does not appear – she wasn't in the
left table
```

6. Pivot Tables: Reshaping Long Data Into Wide Summaries

`df.pivot_table()` reshapes 'long' row-per-observation data into a 'wide' summary table with one dimension as rows, another as columns, and an aggregated value filling the grid — exactly like a spreadsheet pivot table. It's effectively a `groupby` on two dimensions at once, presented as a matrix instead of a list, which makes cross-tabulations (e.g. 'total sales by region and by month, side by side') dramatically easier to read than the equivalent long-format `groupby` output.

PYTHON

```
import pandas as pd

df = pd.DataFrame({
    "region": ["North", "North", "South", "South"],
    "month": ["Jan", "Feb", "Jan", "Feb"],
    "sales": [100, 120, 200, 180],
})

pivot = df.pivot_table(index="region", columns="month", values="sales",
                        aggfunc="sum")
print(pivot)
```

	month	Feb	Jan
region			
North		120	100
South		180	200

Common Pitfalls

- Writing an explicit Python for-loop over a NumPy array or pandas column instead of a vectorized operation — often 10-100x slower for no benefit.
- Using Python's ``and`/`or`` instead of ``&`/`|`` when combining boolean conditions on pandas Series, which raises a confusing `ValueError`.
- Forgetting parentheses around each condition in a combined boolean filter (`df[df.a > 1 & df.b < 2]` instead of `df[(df.a > 1) & (df.b < 2)]`) — operator precedence makes this fail silently or error.
- Using the wrong join type (especially defaulting to `inner` when `left` was intended) and silently losing rows you expected to keep.
- Not checking for duplicate keys before merging, which can silently multiply row counts (a many-to-many merge) far beyond what was intended.
- Assuming broadcasting will 'just work' between arrays without checking shape compatibility, then being confused by a `ValueError` about mismatched shapes.

Key Takeaways

- Vectorized NumPy/pandas operations should replace almost every explicit Python loop over array or column data — both for speed and readability.
- Broadcasting lets operations between differently-shaped arrays work automatically, as long as shapes are compatible from the right.

- Boolean indexing (`df[condition]`) is pandas' declarative equivalent of a SQL WHERE clause — use `&` / `|` with parenthesized conditions.
- `groupby` + `agg` is the split-apply-combine pattern behind almost every summary table you'll ever build from transactional data.
- `merge()`'s `how` parameter is not a detail to skip past — `inner/left/right/outer` produce meaningfully different, easy-to-get-wrong results.

Learn More

NumPy — Broadcasting Documentation

<https://numpy.org/doc/stable/user/basics.broadcasting.html>

pandas — GroupBy: Split-Apply-Combine User Guide

https://pandas.pydata.org/docs/user_guide/groupby.html

pandas — Merge, Join, Concatenate User Guide

https://pandas.pydata.org/docs/user_guide/merging.html